Product Requirement Document

AI-Powered Codebase Ontology & Visualization Platform

Annotated Edition v2 — includes System Architecture and CI/CD Test Impact Analysis Addendum

Table of Contents

1. Project Overview & Core Value Proposition

Modern development teams face massive friction when onboarding engineers, managing context switching, and aligning technical architecture with product objectives. Current tooling treats repositories as flat file structures or disconnected documentation.

This platform is an AI-powered, context-aware codebase ontology and visualization web application. By parsing codebases into interactive, multi-dimensional knowledge graphs, the platform translates raw source code into tailored "tree maps" for different organizational stakeholders (technical, product, onboarding). It solves the "kitchen without labels" dilemma by mapping explicit code dependencies, task management tickets, ownership history, and required skills directly onto a visual graph canvas.

1.1 Target Audience

New Hire Engineers — Accelerate time-to-productivity by visualizing branch-specific context, architectural dependencies, and onboarding workflows.

Technical Leads & Architects — Maintain a clear structural map of code semantics, identify ownership, track style adherence, enforce modular boundaries.

Product Managers — Track feature shipping progress and map Jira/Linear tasks directly to real-world code components.

1.2 Macro View: Multi-Repo Architecture

Before any file tree renders, the platform supports a highest-level, multi-repo view of how work is actually organized across an engineering org. This Macro View is its own graph layer, sitting above the Unified File Skeleton (Section 4.3), and is what a viewer lands on before drilling into a specific repo's Technical/Product/Onboarding Tree.

Project-to-Repo Mapping — A Project is a top-level organizational entity that can map to a single repository or span several (e.g., a "Checkout" project spanning a frontend repo, a backend-api repo, and a shared-libs repo). The Macro View renders Project nodes connected to one or more Repo nodes before any folder or file is shown.

Inter-Repo Context — Distinct repositories are connected by cross-repo edges (e.g., a frontend repo's calls into a backend-api repo, or a shared dependency on a common library repo), inferred from detected API contracts, shared package versions, and cross-repo import statements. This lets a viewer see how repos relate to one another before opening any single repo's file tree.

Selecting a Repo node in the Macro View drills down into that repo's Unified File Skeleton, preserving the same Technical/Product/Onboarding view toggle described in Section 2.2. See Section 4.3 for the Project/Repo node and edge additions to the graph schema.

2. Functional Requirements — with Required Tooling

Each capability below is tagged with the concrete languages, libraries, and services needed to implement it.

2.1 Unified Graph-Based Repository Parsing

Multi-Language AST Parsing — Map explicit structural relationships (function calls, class inheritance, module imports) via incremental syntax tree analysis. Tooling: Tree-sitter, Python, language grammars (JS/TS/Py/Go/etc.).

Semantic Relationship Extraction — Analyze code semantics, config files, and inline documentation via LLMs to extract high-level functional groupings. Tooling: Graphify, LangChain/LangGraph, Anthropic API (Claude), Python/FastAPI.

Delta Processing — Process code deltas incrementally on every push rather than rebuilding the entire codebase graph from scratch. Tooling: GitHub Webhooks, Python, Queue (SQS/Redis Streams).

2.2 Multi-View Interactive Tree Canvas

Canvas Node Model — Regardless of which view is active, the rendered canvas is built from exactly two node types: Folder ("F") and File (including API endpoint files). Functions and classes are parsed and stored as graph data — used to compute IMPORTS/CALLS/INHERITS edges and node-level metadata — but are never surfaced as their own canvas nodes, so the visual graph stays legible at repo scale instead of cluttering down to function/class granularity.

View	Target Users	Scope	Tooling
Technical Tree	Architects, Core Devs	Syntax hierarchies, data models, infra nodes, imports, API routing paths.	React Flow, Cytoscape.js/Sigma.js (WebGL), TypeScript
Product Tree	PMs, UX	High-level feature groupings mapping files to user-facing capabilities.	React Flow, Next.js, Tailwind CSS
Onboarding Tree	New Hires, DevOps	Sequenced setup workflows, dependencies, required dev environments per service/branch.	React Flow, Next.js API routes, shell script execution sandbox

2.3 Context & Metadata Labeling Engine

Ownership & Git History — Automatically map previous contributors, recent editors, and primary code owners to specific nodes based on Git history. Tooling: GitHub REST API, git log parsing, Node.js/TypeScript.

Coding Style & Guidelines — Tag branches and folders with localized architectural expectations, styling rules, and design patterns. Tooling: LLM classification (Claude API), PostgreSQL metadata store.

Onboarding Automation — Embed branch-specific onboarding tasks, pre-requisite reading, and executable local environment terminal setup commands inside node sidebars. Tooling: Next.js, Node.js sandboxed exec (or containerized runner), PostgreSQL.

2.4 Asynchronous Tooling Integrations

GitHub Integration — Native GitHub App authorization capturing live commits, pull requests, and branch creations via webhooks to keep tree structures continually synchronized. Tooling: GitHub Apps/OAuth, Webhooks, Node.js/TypeScript.

Task Management Integration — Bidirectional syncing via Jira REST API and Linear API to dynamically map open issue tickets, bug tracking, and user stories onto relevant feature trees or file nodes. Tooling: Jira REST API, Linear API (GraphQL), Node.js/TypeScript, Queue worker (BullMQ).

Cursor Origin Compatibility (NEW, forward-looking) — Provider-agnostic git integration layer designed to accept Cursor Origin as an additional git-hosting source once its API/webhook surface is public. See Section 6.

2.5 Persistent Memory & Token-Optimized RAG Architecture

Graph-Guided Retrieval — Use the repository graph to pinpoint localized dependencies; the AI only retrieves code from nodes adjacent to the active file/branch, avoiding whole-repo dumps. Tooling: LangGraph, Neo4j/Amazon Neptune Cypher queries, Python/FastAPI.

Hybrid Storage Engine — Split long-term memory across a transactional DB (users/permissions), a graph DB (codebase topology), and a vector store (chunked semantic lookups). Tooling: Amazon RDS (PostgreSQL), Neo4j/Amazon Neptune, pgvector/Amazon OpenSearch.

3. Technical Stack & Infrastructure Architecture — Summary

Layer	Choice	Purpose
Frontend Framework	Next.js (React) + TypeScript	Visualization web app shell
Graph Rendering	React Flow + Cytoscape.js/Sigma.js (WebGL)	Node-based canvas at scale (1000s of nodes)
Styling	Tailwind CSS	Dashboard layouts, context inspectors
Core API	Node.js/TypeScript (Next.js API routes)	Auth, permissions, webhooks
AI / Knowledge Graph Engine	Python FastAPI + Graphify + Tree-sitter + LangChain/LangGraph	Async repo parsing & semantic extraction
Relational DB	Amazon RDS (PostgreSQL)	Orgs, users, permissions, billing, logs
Graph DB	Amazon Neptune or Neo4j (distributed)	Repository ontology (files, authors, tasks, commits, tests)
Vector DB	pgvector or Amazon OpenSearch	Semantic code/doc chunk retrieval

4. System Design Architecture

This section defines how the platform is actually built: the services, data stores, and flows that implement the functional requirements above. Three principles drive every decision here:

- The graph is the source of truth, not the repo. Once ingested, all reads (canvas rendering, AI retrieval, integrations) hit the graph/vector/relational stores — never the raw git checkout.
- The write path and read path are fully decoupled. Webhooks, parsing, and LLM enrichment happen asynchronously in workers; the frontend never blocks on parsing.
- Every persona view (Technical / Product / Onboarding Tree) is a projection, not a separate dataset. One graph schema; three query/filter/layout strategies over the same nodes and edges.

4.1 Logical Architecture

At a high level: the browser client (Next.js + React Flow/Sigma.js) talks to an API Gateway that enforces auth and RBAC, which routes to a Core App Service (Node.js) for fast synchronous reads/writes, and to a Realtime Service (WebSocket + Redis pub/sub) for live graph updates. Heavy work — parsing, semantic enrichment, AI retrieval — runs asynchronously in a Python/FastAPI worker fleet, coordinated through a message bus (SQS/Redis Streams/BullMQ). Three data stores back everything: PostgreSQL (orgs, users, permissions, tickets), a Graph DB — Neo4j or Neptune — (the codebase ontology), and a Vector Store — pgvector or OpenSearch — (semantic code/doc chunks).

4.2 Component Breakdown

Client — Next.js Visualization Shell

- React Flow renders small/medium graphs (<~500 visible nodes); Cytoscape.js/Sigma.js (WebGL) takes over above that threshold — chosen automatically based on a node-count signal from the backend, not a user toggle.
- The Technical/Product/Onboarding tabs re-issue a graph query with a different view parameter rather than maintaining three separate in-memory graphs.
- Viewport-scoped querying: the client requests only nodes in the current viewport plus a one-hop buffer, not the whole graph.
- Live updates arrive over a WebSocket subscription per open project, patching the canvas in place instead of triggering a refetch.

API Gateway / Edge Layer (Node.js)

- AuthN via JWT/session validation; AuthZ middleware filters every request through RBAC before it reaches any data service.
- Exposed as a GraphQL or REST facade, since client data needs vary significantly by view.

Core App Service (Node.js/TypeScript)

- Handles org/user/permission management, node sidebar metadata assembly, and the Graph Query Service that translates viewport/view-filter requests into Cypher/Gremlin queries with RBAC-based node stripping.
- Kicks off async jobs onto the message bus rather than doing heavy work inline.

Realtime Service

- A WebSocket gateway backed by Redis pub/sub for fan-out across multiple gateway instances; every push is re-filtered through the receiving client's RBAC scope before delivery.

Async Worker Fleet (Python + FastAPI)

Ingestion/Delta Worker — clones or diffs the repo, runs Tree-sitter (incremental on delta pushes, full parse only on initial onboarding), writes node/edge upserts to the Graph DB, updates embeddings in the Vector Store, publishes a graph.updated event.

Semantic Enrichment Worker — consumes newly parsed nodes on a separate queue so LLM latency never blocks structural sync; calls Graphify + LangChain/LangGraph + the Claude API to build feature clusters (Product Tree), style tags, and onboarding step summaries.

Retrieval Service (Graph-Guided RAG) — resolves an anchor node, traverses a bounded number of hops across structural edges, pulls only the Vector Store chunks for that subgraph, and calls the Claude API with a bounded context window — the mechanism behind the 70x token-reduction target.

Integration Workers (Node.js)

- GitHub sync: webhook receiver validates signatures, normalizes events onto the message bus, and updates ownership/PR/commit metadata directly on Graph DB node properties.
- Jira/Linear sync: scheduled + webhook-driven bidirectional sync via a BullMQ-backed worker, mapping ticket IDs to file/feature nodes.

4.3 Data Model

Relational Store (PostgreSQL) — who, permissions, config

Owns organizations, users, repositories, permissions (scoped by org/repo/branch/path), a ticket cache linking to graph node references, style rules, and onboarding steps. Postgres is the system of record for permissions specifically, so RBAC decisions are always computed against one auditable source.

Graph Store (Neo4j / Amazon Neptune) — the ontology

Node labels: Project, Repo, Folder, File (including API endpoint files), Function, Class, Module, Person, Commit, Feature (LLM-derived), Ticket (mirror pointer), EnvStep (onboarding), and Test (new — see Section 5). Only Folder and File nodes are ever rendered on the canvas, in any of the three tree views; Function and Class remain graph-internal, used to derive structural edges and node metadata rather than to draw their own boxes (Section 2.2). Edge types: CONTAINS (Project→Repo, Folder→Folder/File), CONNECTS_TO (cross-repo, Macro View — Section 1.2), IMPORTS, CALLS, INHERITS, OWNS, AUTHORED, MODIFIED, REFERENCES, PART_OF, REQUIRES, DEPENDS_ON, and COVERS (new — links Test nodes to the Function/File nodes they exercise).

The three tree views are query shapes over this one schema: Technical Tree = Folder/File nodes plus their structural edges (IMPORTS/CALLS/INHERITS, computed from the underlying Function/Class data); Product Tree = Feature nodes plus PART_OF/REFERENCES edges, with dashed-boundary grouping over the same Folder/File nodes

(Section 4.7); Onboarding Tree = EnvStep nodes ordered by sequence. The Macro View (Section 1.2) sits above all three as a Project/Repo-level projection of the same graph.

Vector Store (pgvector / OpenSearch) — semantic chunks

Code and doc chunks are embedded and keyed back to their originating graph node ID, namespaced per organization, so Graph-Guided Retrieval can fetch only the chunks belonging to a graph-bounded candidate set rather than running a similarity search over the whole corpus.

4.4 Key Flow Walkthroughs

Initial Repository Onboarding (cold start)

User authorizes the GitHub App, a full-parse job runs once across the whole repo (Tree-sitter parse, bulk graph write, full embedding pass), then Semantic Enrichment runs feature clustering, style tagging, and onboarding-step generation across the repo. The Technical Tree becomes available first; Product/Onboarding trees may briefly show a "generating" state.

Steady-State Push Event (the 15-second sync target)

A push triggers a webhook, the Ingestion Worker parses only the changed files, diffs the AST against the existing graph, writes upserts/deletes in one transaction, and publishes a graph.updated event that the Realtime Service fans out over WebSocket — the canvas patches only the affected nodes. Semantic re-enrichment for the changed nodes happens in parallel, slightly after, and is explicitly out-of-band from the 15-second budget.

AI Query (Graph-Guided RAG)

A user question against a node routes to the Retrieval Service, which resolves the anchor node, traverses a bounded number of hops, fetches only the in-scope Vector Store chunks, assembles a bounded prompt, and calls the Claude API — logging context tokens used versus a hypothetical full-repo token count to make the 70x reduction KPI measurable rather than aspirational.

4.5 Security & Multi-Tenancy

- Tenant isolation is structural, not just a WHERE clause: org_id is stamped on every graph write and injected server-side on every query; the vector store uses per-tenant namespaces/indices; Postgres uses row-level security as defense-in-depth.
- Granular (node-level) access control is enforced centrally in the Graph Query Service — unauthorized nodes are filtered server-side before leaving the server, and the same filter applies to WebSocket fan-out.
- Data ephemerality: the Ingestion Worker operates against a scratch clone destroyed after parsing; only structural metadata and embeddings persist, never raw source trees.
- Secrets (GitHub App keys, Jira/Linear OAuth tokens) live in a KMS-backed secrets manager; webhook payloads are signature-validated before being trusted onto the message bus.

4.6 Success Metrics Mapped to Architecture

Metric	Target	Architectural Lever
Token Optimization Rate	≥ 70x reduction vs. full-repo context	Graph-Guided Retrieval structurally bounds context to a traversal-derived candidate set.
Time-to-First-Commit (TTFC)	Reduce by 40%+	Onboarding Tree generation prioritized on initial ingestion via its own priority queue.
Sync Latency	Canvas updates within 15s of webhook	Delta-only parsing, decoupled LLM enrichment, WebSocket push instead of polling.

4.7 Feature Cluster Visualization & Metadata Panel

Feature Clusters render as dashed boundary lines drawn around the set of Folder/File nodes the Semantic Enrichment Worker groups into a given Feature node, independent of folder structure — a single feature routinely spans files in unrelated directories (e.g., an API route file, a shared util, and a frontend component). Clicking a boundary opens a metadata panel with four fields:

Description — an AI-generated summary of the feature's user-facing capability, produced by the Semantic Enrichment Worker (Section 4.2) and cached alongside the Feature node.

Team — the owning pod/group, resolved from a CODEOWNERS-style mapping or a manually assigned override.

Files Included — the live list of File/Folder nodes currently inside the dashed boundary, kept in sync via the PART_OF edge whenever the Ingestion Worker's delta pass adds or removes files from the feature.

Recent Edits — Git history and recent pull requests scoped to just the files inside the boundary, pulled from the same GitHub sync pipeline described in Sections 2.4 and 4.2, rather than a repo-wide commit feed.

4.8 Interactive Node Overlay Panel (Node-Level Toggles)

Each Folder and File node exposes a set of independently toggleable overlay layers in its sidebar/tooltip, so a viewer sees only the data relevant to their current question rather than every property at once. Toggling a layer re-renders just that panel section; it does not re-query the graph.

Dependencies — auto-extracted tool requirements (e.g., package.json/requirements.txt/go.mod data) merged with manually added tech-lead prerequisites, each rendered as an actionable download link or copyable setup command.

Recent Changes — Git metadata — recent editors, commit messages, linked PRs — scoped to that specific node, sourced from the ownership pipeline in Sections 2.3/4.2.

Specific Features — tags naming which Feature Cluster(s) (Section 4.7) this node belongs to, with a click-through back to that cluster's dashed boundary.

Agent / Skill MD — metadata describing the developer skills or AI-agent operating parameters (allowed tools, required context, guardrails) needed to safely modify this node, authored as node-scoped Markdown — analogous to a per-file AGENTS.md/SKILL.md — and surfaced to both human viewers and the platform's own Retrieval Service (Section 4.2) before it proposes a change.

Description — a natural-language summary of what the file/folder does, generated by the Semantic Enrichment Worker.

Specific Tickets — Jira/Linear tickets mapped to the node via the ticket-to-node references already in the relational store (Section 4.3), surfaced without leaving the canvas.

4.9 Folder-Level Topological Inheritance

To avoid re-tagging every file in a domain, two rule types are attached to Folder ('F') nodes and cascade down to every nested child File by default, overridable at a deeper folder or file level:

Coding Style — architectural guidelines and localized syntax rules (naming conventions, preferred patterns) tagged once on a Folder node and inherited by all files beneath it, via the same LLM classification pipeline described in Section 2.3.

Comment Style — documentation and inline-comment formatting expectations (docstring format, required header blocks) inherited the same way, tracked as a rule set separate from Coding Style so the two can diverge independently — a folder can inherit strict comment requirements without inheriting a different folder's naming convention.

Inheritance resolves top-down at query time in the Graph Query Service (Section 4.2): a file's effective style is its nearest ancestor Folder's rule unless the file itself carries an override, so style data is stored once per Folder rather than duplicated onto every File node.

6. Forward-Looking Integration: Cursor Origin

Cursor announced Origin, an agent-native git hosting platform, on June 16, 2026, positioned as a git-compatible alternative to GitHub built around AI agents as primary users of version control rather than humans. As of this writing it remains waitlist-only, with general availability targeted for fall 2026, and no public API or webhook documentation has been released.

6.1 Why It's Relevant to This Product

- Origin is described as git-compatible and extensible via API and the Model Context Protocol (MCP), which suggests it should be integrable through the same abstraction already used for GitHub — a webhook-driven event source feeding the Ingestion Worker — rather than requiring a parallel subsystem.
- Its MCP extensibility is notable because the platform's Semantic Enrichment and Retrieval services could potentially interact with Origin as an MCP server directly, rather than through a bespoke REST integration.

6.2 Recommended Approach

- Design the git Integration Worker generically now — provider field, normalized event schema, pluggable auth — so a second provider can be added without refactoring the ingestion pipeline.
- Do not build a concrete Origin integration until its API/webhook documentation is public; treat this as a placeholder integration point tracked for fall 2026, not a committed sprint item.
- Re-evaluate once Origin ships public docs, particularly around how it exposes commit/branch events and whether its MCP surface can replace a portion of the custom webhook-handling code.

7. Development Roadmap — Updated Sprint Plan

The original six-sprint (~12 week) MVP plan is preserved; the CI/CD Test Impact feature is folded into Sprints 4-5 alongside the existing Jira/Linear and RBAC work, and Cursor Origin is tracked as a post-MVP watch item rather than a committed sprint deliverable.

Sprint	Weeks	Goal	New items added
Sprint 0	1-2	Foundations & infra skeleton	Add Test node type and COVERS edge type to the initial graph schema definition.
Sprint 1	3-4	Repository parsing pipeline	Parse test files as Test nodes during Tree-sitter ingestion; static COVERS edge inference from imports.
Sprint 2	5-6	Technical Tree Canvas (MVP UI)	No change.
Sprint 3	7-8	Semantic layer & Product Tree	No change.
Sprint 4	9-10	Integrations: GitHub, Jira, Linear	design git Integration Worker to be provider-generic in anticipation of Cursor Origin.
Sprint 5	11-12	Onboarding Tree, permissions & hardening	Add dynamic COVERS edge refresh job (coverage-instrumented nightly run); pass/fail badge overlay on Technical Tree; safe-fallback-to-full-suite logic.

Note: the Canvas Node Model (Folder/File only — Section 2.2) and the Folder-Level Topological Inheritance rules (Section 4.9) are schema-level and belong in Sprint 0/1 alongside the initial graph schema. The Feature Cluster metadata panel (Section 4.7) lands with Product Tree in Sprint 3. The Interactive Node Overlay panel (Section 4.8) is UI-only over data that already exists by Sprint 4 (ownership, style, tickets, test coverage), so it is scoped into Sprint 4-5 alongside the other node-sidebar work. The Macro View (Section 1.2) requires a Project entity and

cross-repo edge inference not otherwise needed for a single-repo MVP, so it is scoped as a Sprint 6+ (post-MVP) addition below rather than a committed MVP sprint deliverable.

Post-MVP (Sprint 6+): expand language coverage (Go, Java, Rust), add more task-tracker integrations (Asana, GitHub Issues), introduce collaborative annotations on the graph, add the Macro View (Project-to-Repo mapping and inter-repo context — Section 1.2), build the admin billing/plan console, and revisit Cursor Origin integration once its public API/webhook documentation ships.